

Documento de Referencia: Innovación, Dificultad, Investigación.

Proyecto: Aplicación “**No Te Cortes**”

Sector: Servicios de Estética e Imagen Personal (CNAE 9602)

0. Análisis Estratégico del Sector y Oportunidad

Este apartado responde a los criterios de identificación de necesidades, oportunidades y tipo de proyecto.

Necesidades más demandadas

Se han identificado las carencias críticas que el software debe resolver:

- **Profesional:** Necesidad de eliminar la "interrupción operativa" (dejar de cortar el pelo para atender el teléfono) y reducir las pérdidas por clientes que no asisten (No-shows).
- **Cliente:** Demanda de inmediatez en la reserva (24/7) y recordatorios automáticos sin necesidad de llamadas.

Valoración de Oportunidades de Negocio

- **Mercado desatendido:** El 53% del sector sigue usando agenda de papel. Existe una oportunidad clara para una herramienta de baja barrera de entrada.
- **Ventaja Competitiva:** Los grandes software (Booksy, Vagaro) son complejos y caros para el micro-autónomo. “**No Te Cortes**” cubre el nicho de la simplicidad y el coste operativo cero.

Tipo de Proyecto Requerido

Para dar respuesta a lo anterior, se requiere un proyecto basado en arquitectura *Serverless* para garantizar escalabilidad y viabilidad económica para el pequeño negocio.

1. Innovación

Clasificación de empresas del sector

A continuación, se clasifican las empresas según sus características organizativas y su encaje con la solución:

Tipo de Empresa	Organización	Producto/ Servicio	"No Te Cortes"
Micro-Salón (Autónomo)	1 empleado (dueño).	Servicios rápidos (Corte/Barba ...).	Máximo: Elimina la interrupción telefónica mientras trabaja.
Salón de	2-3	Servicios largos	Alto: Gestión de bloques

Tipo de Empresa	Organización	Producto/ Servicio	"No Te Cortes"
Estética Especializado	empleados.	(Tinte/Tratamientos).	de tiempo y servicios específicos.
Franquicia / Gran Salón	+5 empleados, recepción.	Gran volumen de clientes.	Medio: Requiere gestión de turnos avanzada (Escalabilidad futura).

"No te cortes" no es solo una agenda digital; es una respuesta estratégica a la polarización del mercado detectada en el estudio previo.

- **Propuesta de Valor Único:** Simplificación extrema del flujo de trabajo frente a la competencia abrumadora.

Mientras que la competencia (Booksy, Vagaro) abruma al pequeño autónomo con funciones innecesarias y costes altos, mi innovación reside en la **extrema simplificación del flujo de trabajo**.

- **Superación de la competencia:** Atacamos la **debilidad del sector (53% sin digitalizar)** eliminando la barrera de entrada económica mediante una arquitectura *serverless* que reduce los costes operativos a casi cero.
- **Innovación Funcional:** La implementación de un sistema de **"Estado de Usuario" (Activo/Inactivo)** permite al administrador un control de seguridad que las apps genéricas (WhatsApp/Google Calendar) no poseen, profesionalizando la gestión sin añadir complejidad.
- **Arquitectura Serverless:** Reducción de costes operativos a casi cero.
- **Innovación Funcional:** Sistema de "Estado de Usuario" para control de seguridad y profesionalización del acceso.

Principales puntos innovadores:

- Gestión de citas totalmente automatizada.
- Reservas sin llamadas ni mensajes.
- Recordatorios automáticos para evitar olvidos .
- Interfaz adaptada a usuarios con poca experiencia digital.
- Sistema ligero basado en Firebase, económico y escalable.

2. Dificultad técnica

La implementación de **"No Te Cortes"** supone un desafío técnico que va más allá de la creación de interfaces, centrándose en la integridad de los datos y la seguridad del sistema. Los componentes más complejos son:

A. Lógica de Concurrencia y Disponibilidad en Tiempo Real

Uso de operaciones atómicas en Firestore para evitar el *overbooking* (dos personas reservando el mismo segundo).

Uno de los mayores retos técnicos en una aplicación de reservas es la **gestión de la concurrencia**. El sistema debe garantizar que dos usuarios no puedan reservar el mismo servicio, con el mismo profesional y a la misma hora exacta de forma simultánea.

- **Solución:** Se ha investigado el uso de **operaciones atómicas** y la lógica de verificación previa en Firestore. Esto asegura que el "hueco" en la agenda se valide en el servidor justo antes de confirmar la reserva, evitando conflictos de agenda (*overbooking*) incluso si las peticiones llegan con milisegundos de diferencia.

B. Arquitectura y Normalización de Datos NoSQL

Diseño de colecciones y subcolecciones optimizadas y uso de `collectionGroup` para consultas globales de historial.

A diferencia de las bases de datos relacionales tradicionales, estructurar la información en **Firestore (NoSQL)** requiere un esfuerzo de diseño previo para garantizar la eficiencia.

- **Desafío:** Organizar citas y horarios en una jerarquía de colecciones y subcolecciones que permita consultas rápidas.
- **Implementación Avanzada:** Se ha implementado la función **`collectionGroup`** para el módulo de historial. Esto permite realizar búsquedas globales a través de múltiples subcolecciones de "horas" distribuidas entre diferentes empleados, una técnica avanzada que optimiza el rendimiento de la app y reduce el consumo de cuotas de lectura.

C. Sistema de Control de Acceso Basado en Roles

Motor de verificación de roles (`isAdmin`) que inyecta interfaces dinámicas según el usuario.

La aplicación debe comportarse de manera distinta según quién la utilice, lo que añade una capa de complejidad en la gestión de la lógica del lado del cliente.

- **Lógica implementada:** Se ha desarrollado un motor de verificación que, tras el *login*, consulta el documento del usuario en la base de datos para identificar los campos `rol` e `isAdmin`.
- **Diferenciación de Interfaces:** Dependiendo de estos atributos, la aplicación inyectará dinámicamente el panel de administración o la interfaz de cliente, protegiendo las funciones críticas de gestión de servicios, precios y bloqueos de fechas frente a usuarios no autorizados.

D. Gestión del Ciclo de Vida Global:

Extensión de la clase Application para políticas de seguridad a nivel de proceso.

Se ha implementado una extensión de la clase Application para establecer una política de seguridad a nivel de proceso. Esto permite que la gestión de errores no dependa de una pantalla específica, sino que sea una característica intrínseca de toda la arquitectura del software.

E. Seguridad Perimetral y Cumplimiento Normativo:

Implementación de *Security Rules* de Firebase para cumplimiento nativo de la RGPD.

La dificultad técnica no solo fue mostrar los datos, sino protegerlos. En un entorno Cloud, el código de la App no es suficiente para garantizar la privacidad. Por ello, **investigué e implementé las Security Rules de Firebase** para asegurar que, aunque los datos viajen por la nube, la seguridad se ejecute a nivel de servidor.

Esto garantiza que un cliente 'A' nunca tenga permisos de lectura sobre las citas de un cliente 'B', cumpliendo así con la **RGPD de forma nativa** y protegiendo la integridad de la base de datos frente a accesos no autorizados desde fuera de la aplicación."

F. Estrategia de Desnormalización de Datos:

Duplicidad estratégica de datos para mejorar la fluidez en dispositivos con poca cobertura.

Una de las mayores dificultades fue la **estrategia de desnormalización**. En bases de datos tradicionales (SQL) los datos no se repiten, pero para que '**No Te Cortes**' sea fluida en móviles con poca cobertura, decidí duplicar ciertos datos (como el nombre del servicio dentro de cada objeto Cita). Esto reduce las consultas a Firebase, ahorra batería al usuario y mejora la velocidad de carga del historial

Componentes complejos necesarios:

- Autenticación segura con Firebase Authentication.
- Base de datos NoSQL en Firestore, sincronizada en tiempo real.
- Notificaciones automáticas programadas con Firebase Cloud Messaging.
- Gestión de horarios dinámicos según servicios y duración.
- Arquitectura Android moderna con fragments, navegación y validación.

3. Trabajo de Investigación

Tecnologías investigadas:

1. **Firestore Authentication:** Gestión segura de identidad.
2. **Firestore Database:** Base de datos NoSQL en tiempo real.
3. **Firestore Cloud Messaging:** Notificaciones push seguras.
4. **Jetpack Navigation:** Gestión centralizada de rutas y fragments.
5. **Material Design 3:** Directrices de UI para baja alfabetización digital.

Se detallan a continuación.

1. **Firestore Authentication.**

Es un servicio de autenticación de usuarios que simplifica la gestión de usuarios en aplicaciones web y móviles. Permite a los desarrolladores gestionar el registro, inicio de sesión y cierre de sesión de los usuarios de forma segura utilizando varios métodos, como correo electrónico y contraseña, o proveedores de redes sociales como Google o Facebook.

2. **Firestore Database.**

Es una base de datos de documentos NoSQL, en tiempo real y escalable, alojada en la nube por Google. Permite almacenar, sincronizar y consultar datos para aplicaciones web y móviles de forma global y segura, organizando la información en colecciones y documentos, en lugar de tablas y filas. Ofrece funcionalidades como la sincronización en tiempo real y la capacidad de trabajar sin conexión.

3. **Firestore Cloud Messaging.**

Firestore Cloud Messaging es un servicio de mensajería gratuito que permite a las aplicaciones enviar y recibir mensajes de forma segura y confiable entre servidores y dispositivos. Facilita el envío de notificaciones push y mensajes dentro de la aplicación a los usuarios de plataformas como iOS, Android y la web, desde la consola de Firestore o a través de un servidor de aplicaciones.

En esta fase se investigó la diferencia entre las **notificaciones push (servidor)** y las **alarmas locales (cliente)**. Se ha priorizado el estudio de **FCM** porque, a diferencia de las alarmas locales que consumen recursos del sistema de forma constante, las notificaciones push permiten que el dispositivo "despierte" solo cuando recibe el mensaje. Esta decisión de diseño es clave para **ahorrar batería** en el terminal del profesional, garantizando que la aplicación sea ligera y no afecte al rendimiento del teléfono durante la jornada laboral.

4. **Jetpack Navigation.**

Es una biblioteca de Jetpack que facilita la navegación entre diferentes destinos (pantallas) dentro de una aplicación de Android. Permite definir de forma centralizada todos los destinos y las rutas posibles, así como gestionar la transición entre ellos.

5. Material Design.

Es un sistema de diseño de Google que establece directrices, componentes y herramientas para crear interfaces de usuario visualmente consistentes, intuitivas y responsivas en múltiples plataformas y dispositivos. Se inspira en principios del mundo físico, como capas de papel, sombras, luces y movimiento, para crear una experiencia digital que se sienta más natural e interactiva.

Gestión de la Programación Asíncrona

- **Modelo Asíncrono:** Uso de objetos Task y Listeners para evitar el bloqueo del *Main Thread*.
- **Patrón MVVM (Model-View-ViewModel):** Decisión estratégica para separar la lógica de negocio de la interfaz, permitiendo que la App sea testable y escalable.

Uno de los hitos más significativos en el desarrollo de este proyecto será el dominio de la **programación asíncrona**, un concepto avanzado que no se profundiza en el temario básico y que resulta crítico para la estabilidad de la aplicación.

- **Android** no permite realizar operaciones de red (como consultar Firebase) en el hilo principal (*Main Thread*), ya que esto congelaría la interfaz de usuario. Por tanto, será necesario investigar y aplicar un modelo de ejecución en segundo plano.
- **Implementación:** Tendré que aprender a gestionar el ciclo de vida de las peticiones mediante el uso de objetos **Task** y la implementación de **Listeners** (como `OnCompleteListener`, `SuccessListener` y `FailureListener`).
- **Dificultad:** Esto implica un cambio de mentalidad en la programación: el código no se ejecuta de forma secuencial, sino que "espera" a que la base de datos responda. Aprender a orquestar estas respuestas para actualizar la interfaz de usuario sin errores de ejecución (*crash*) demuestra un nivel de competencia técnica que garantiza una experiencia de usuario fluida y profesional.
- Aunque la funcionalidad de reserva se menciona en el análisis de necesidades, su implementación técnica supuso el reto de asegurar la **Atomicidad**. Tuve que investigar cómo Firestore gestiona las escrituras para que, si falla la conexión a internet en mitad de una reserva, el sistema no quede en un estado inconsistente.

Patrones de Diseño: Arquitectura MVVM

Durante el proceso de investigación, se descartó el modelo de desarrollo lineal por ser poco escalable. En su lugar, se profundizó en el patrón **MVVM (Model-View-ViewModel)**.

- **Innovación en el flujo de datos:** Este aprendizaje ha permitido separar la lógica de negocio (consultas a Firestore) de la lógica de la interfaz. De este modo, la aplicación es mucho más fácil de mantener y permite que la interfaz se "suscriba" a

los cambios de los datos, reaccionando automáticamente ante actualizaciones en tiempo real.

- La elección de **MVVM** (Model-View-ViewModel) no es solo una preferencia estética, sino una decisión estratégica. Permite que la aplicación sea **testable**. Si en el futuro el salón quiere cambiar los precios, la lógica reside en el ViewModel y no hay que reprogramar todas las pantallas de la aplicación."

Tecnologías nuevas aprendidas durante el proyecto:

1. Bases de datos NoSQL.

Son un tipo de base de datos no relacional que no utiliza el modelo tabular de tablas con filas y columnas, sino que almacena datos en formatos más flexibles como documentos, clave-valor, grafos o columnas.

2. Notificaciones programadas.

Son mensajes automáticos que se envían a una fecha y hora específicas, en lugar de hacerlo inmediatamente

3. Arquitectura MVVM en Android.

Es un patrón de diseño para aplicaciones de Android que separa las responsabilidades entre la interfaz de usuario (View), la lógica de presentación (ViewModel) y la lógica de negocio (Model).

4. Sincronización en tiempo real.

Es el proceso de actualizar instantáneamente la información en múltiples dispositivos o sistemas en cuanto se produce un cambio, para que todos tengan acceso a la versión más reciente de los datos. Esto permite que las modificaciones sean visibles de inmediato en toda la red, asegurando consistencia y acelerando la respuesta a cambios, y es una tecnología fundamental para aplicaciones como el chat o los juegos en línea, así como para la gestión de inventarios empresariales.

5. Gestión de ciclo de vida de fragments.

La gestión del ciclo de vida de un fragmento en Android se refiere a la secuencia de estados (como onCreate, onStart, onResume, onPause, onStop, onDestroy) por los que pasa un fragmento desde su creación hasta su destrucción. Estos estados son controlados por la actividad principal que lo aloja y permiten a un fragmento gestionar su interfaz de usuario y sus eventos de entrada de forma independiente. Un fragmento puede ser agregado o eliminado dinámicamente mientras la actividad está en ejecución, pero su estado está directamente ligado al de la actividad que lo contiene.

4. Aclaraciones

Este informe técnico-estratégico no cubre por si solo los puntos:

- Clasificación del sector

Han sido tratados en el punto anterior "*Análisis de sector*".

Aunque el envío de notificaciones mediante **Firestore Cloud Functions** está en fase de estudio de costes, la arquitectura ha sido diseñada para ser **compatible con ganchos (hooks) de servidor**. Esto garantiza que la aplicación sea **financieramente viable** para el micro-autónomo, manteniendo un modelo de 'pago por uso' que escala solo si el negocio crece."

5. Fuentes de Información y Referencias

Para la resolución de los desafíos técnicos y la definición de la arquitectura de "No Te Cortes", se han consultado las siguientes fuentes especializadas:

1. **Android Developers:** Guías oficiales de Arquitectura (MVVM) y Navigation Component.
2. **Google Firebase Documentation:** Documentación técnica sobre transacciones atómicas y reglas de seguridad.
3. **Firebase Blog:** Estrategias de modelado NoSQL y desnormalización.
4. **Material Design 3 Guidelines:** Estándares de componentes y UX.

A. Arquitectura y Patrones de Desarrollo

- **Android Developers - Guide to App Architecture:** Documentación oficial de Google sobre el patrón **MVVM**. Fuente fundamental para separar la lógica de negocio de la interfaz y asegurar la "testabilidad" del software.
- **Jetpack Navigation Component:** Guía técnica para la implementación de grafos de navegación, esencial para reducir la complejidad del ciclo de vida de los fragments en el MVP.

B. Persistencia y Lógica de Datos

- **Google Firebase Documentation - Cloud Firestore:** * Transactions and Batched Writes: Fuente clave para resolver el problema de la **Atomicidad** y evitar el overbooking en las reservas simultáneas.
 - Data Bundles and Indexing: Consulta técnica para la implementación de **Collection Group Queries**, permitiendo la búsqueda eficiente en el historial del usuario.
- **NoSQL Data Modeling (Firebase Blog):** Estrategias de **desnormalización** para optimizar el rendimiento en dispositivos móviles y reducir el consumo de cuotas de lectura/escritura (coste cero).

C. Seguridad y Cumplimiento Legal

- **Firebase Security Rules Reference:** Manual de sintaxis para el lenguaje de reglas de Firebase, utilizado para garantizar que el acceso a los datos cumple con la **RGPD** de forma nativa en el servidor.
- **Google Cloud Privacy & Security:** Estándares de seguridad de la infraestructura **Serverless**, justificando la viabilidad del proyecto frente a ataques externos o pérdida de integridad de los datos.

D. Metodología de Desarrollo y UX

- **Material Design 3 (M3) Guidelines:** Documentación sobre componentes de interfaz para asegurar que la app sea intuitiva para usuarios con baja alfabetización digital (objetivo del MVP).
- **Documentación de Firebase Authentication:** Protocolos de seguridad para la gestión de tokens de usuario y persistencia de sesión segura.